

Especificação Técnica de Arquitetura

Sistema de Atualização Automatizada de ERP (V4)

Agosto de 2026 | Abordagem Poliglota & Extração Universal

1. Visão Geral Arquitetural

Este documento estabelece o projeto técnico definitivo para a automação de atualizações do ecossistema ERP. O sistema substitui as atualizações manuais nos servidores por uma arquitetura distribuída (Nuvem + Agente Local), garantindo a integridade do banco `JUNIOR.fdb` e a distribuição autônoma de binários via `BEXE.fdb`.

Stack Tecnológica Definida:

- **API Central (Nuvem):** Python (FastAPI) ou Node.js. Alta performance para filas e distribuição de payloads estáticos.
- **Agente Atualizador (Servidor Local):** C# (.NET 8 Worker Service). Estabilidade absoluta no Windows, threads assíncronas seguras e driver Firebird robusto.
- **Interfaces e Scripts:** Delphi (Mantém o código legado do `b_vendas` e `BScript.exe` intactos).

2. O Banco de Dados de Produção (JUNIOR.fdb)

O Agente operará de forma isolada, mas o ERP precisa ter o poder de "aprovar" o momento da atualização. Para isso, usaremos o banco principal como mensageiro.

2.1. Estrutura de Controle (DDL)

Executar no banco `JUNIOR.fdb` em todos os clientes:

```
CREATE TABLE SYS_ATUALIZACAO (  
    ID INTEGER NOT NULL PRIMARY KEY,  
    VERSAO_NOVA VARCHAR(30) NOT NULL,  
    STATUS VARCHAR(20) NOT NULL,  
    -- 'PENDENTE', 'AUTORIZADO', 'PROCESSANDO', 'CONCLUIDO', 'ERRO'  
    DATA_DOWNLOAD TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
    DATA_AUTORIZACAO TIMESTAMP,  
    MENSAGEM_LOG VARCHAR(500)  
);  
  
INSERT INTO SYS_ATUALIZACAO (ID, VERSAO_NOVA, STATUS) VALUES (1, '0.0.0', 'CONCLUIDO');
```

3. A API Central (Nuvem)

A API atua como um maestro. Ela deve possuir um banco de dados próprio (ex: PostgreSQL) mapeando qual CNPJ tem direito a qual módulo (ex: BNFE, B_Importa).

3.1. Rota de Polling (GET)

Endpoint: `/api/update/check/{cnpj}?versao={versao_atual}`

Lógica: O Agente (C#) bate nesta rota diariamente. A API valida o CNPJ, monta um *payload* com os links para download dos arquivos (.7z ou pacotes DelphiXE) específicos daquele cliente e retorna.

```
{
  "update_available": true,
  "version": "2026.08.10",
  "packages": [
    {
      "file": "B_VENDAS-production-20260810.7z",
      "url": "https://cdn.suaapi.com/B_VENDAS-production-20260810.7z",
      "md5": "a3c4f9e1..."
    },
    {
      "file": "Desktop_DelphiXE_B-NFE_07082026.rar",
      "url": "https://cdn.suaapi.com/Desktop_DelphiXE_B-NFE_07082026.rar",
      "md5": "b8f3e2a9..."
    }
  ],
  "script_url": "https://cdn.suaapi.com/BScript_v4.exe"
}
```

4. O Agente Atualizador (.NET 8 Worker Service)

O Serviço Windows, escrito em C#, é o coração da operação. Ele é auto-contido (não requer instalação de frameworks no servidor do cliente) e opera no diretório principal do ERP.

Motor de Extração Universal (Crucial)

O repositório de atualizações contém pacotes compactados nativamente em **.7z** e pacotes legados nomeados como **Desktop_DelphiXE_...** (compressão WinRAR/ZIP). Para evitar a criação de parsers complexos, a equipe C# **deve** embutir o executável `7za.exe` (7-Zip Standalone) no diretório do Agente.

O 7-Zip ignora extensões e lê o cabeçalho binário do arquivo. O Agente executará via `Process.Start()` o comando: `7za.exe x pacote_baixado.* -oC:\PastaTemp -y`, extraindo qualquer formato com 100% de precisão.

4.1. Máquina de Estados (Workflow do Serviço C#)

Estado 1: Download e Preparação Silenciosa

1. O **Worker** consulta a API. Se ``update_available == true``, inicia o download assíncrono.
2. Compara o hash MD5 dos arquivos baixados com o JSON da API.
3. Aciona o `7za.exe` para extrair todos os executáveis para uma pasta `\TempUpdates`.
4. Conecta-se ao `JUNIOR.fdb` via `FirebirdSql.Data.FirebirdClient` e executa:

```
UPDATE SYS_ATUALIZACAO SET STATUS = 'PENDENTE', VERSAO_NOVA = @v WHERE ID = 1;
```

Estado 2: O Gatilho no ERP (Delphi `b_vendas`)

- Sempre que o ERP (`b_vendas`) for **iniciado** (ex: no evento `OnCreate` do formulário principal ou na tela de login), ele fará um `SELECT` na tabela `SYS_ATUALIZACAO`.
- Ao constatar `STATUS = 'PENDENTE'`, o ERP avisa o administrador imediatamente em tela.
- Ao confirmar, o ERP altera o status para `AUTORIZADO` e encerra-se (`Application.Terminate`).

Estado 3: Execução Crítica (Servidor)

O Agente C#, que monitorava a tabela a cada 10 segundos, detecta `AUTORIZADO` e assume o controle exclusivo do servidor seguindo **estritamente** esta ordem:

Ordem	Processo	Comando C# (Process.Start) / Instrução
1	Bloqueio GFIX	<code>gfix.exe -user SYSDBA -password masterkey -shut force_0 JUNIOR.fdb</code>
2	Backup (BKP_1)	<code>gbak.exe -b -user SYSDBA -password masterkey JUNIOR.fdb JUNIOR_PRE.fbk</code>
3	Scripts (DDL)	<code>BScript.exe /silent</code> (O Agente deve usar <code>Process.WaitForExit()</code> para aguardar o fim).
4	Backup (BKP_2)	<code>gbak.exe -b -user SYSDBA -password masterkey JUNIOR.fdb JUNIOR_POS.fbk</code>
5	Injeção BEXE	O Agente C# conecta no <code>BEXE.fdb</code> . Lê os novos <code>.exe</code> extraídos como byte array (<code>File.ReadAllBytes</code>) e injeta no campo BLOB via comando <code>UPDATE</code> .
6	Liberação GFIX	<code>gfix.exe -user SYSDBA -password masterkey -online JUNIOR.fdb</code>

Estado 4: Finalização

1. O Agente marca `STATUS = 'CONCLUIDO'` no JUNIOR.fdb.
2. Deleta os arquivos da pasta temporária.
3. Envia um HTTP POST (Webhook) para a API Central informando o sucesso para o Dashboard de Monitoramento da nuvem.

5. Tratamento de Falhas (Segurança Firebird)

Devido à natureza não-transacional do Firebird 2.5 para metadados (DDL), o Agente atua como salvaguarda absoluta:

- Se o `gbak` (BKP_1) retornar erro (`ExitCode != 0`), o fluxo é abortado. O banco é devolvido online e a API notificada.
- Falhas no `BScript.exe` não estruturais (como tentar criar campo existente) são inerentes ao seu design idempotente e não devem travar a finalização, exceto em falha de conexão de banco relatada no log gerado.

Ganhos Arquiteturais com a Stack Definida:

A migração da responsabilidade local do Delphi para um Serviço .NET elimina gargalos de performance no download, problemas com bibliotecas de internet obsoletas, resolve a descompressão heterogênea de arquivos, e garante um fluxo de orquestração de processos (gfix/gbak) limpo e altamente monitorável na nuvem.